

```
In [3]: 1 class Human:
2         species="Homo Sapiens"
3         def __init__(self,name,age,gender):
4             self.name=name
5             self.age=age
6             self.gender=gender
```

```
In [4]: 1 x=Human('Parth',32,"Male")
2         y=Human('Aakash',21,'Male')
3         print(x.name)
4         print(y.age)
```

Parth
21

```
In [7]: 1 del Human
```

```
In [8]: 1 x=Human('Parth',32,"Male")
```

```
-----
NameError                                Traceback (most recent call last)
Input In [8], in <cell line: 1>()
----> 1 x=Human('Parth',32,"Male")

NameError: name 'Human' is not defined
```

```
In [5]: 1 class Employee:
2         def __init__(self):
3             print('Employee Class created')
4         def __del__(self):
5             print('Destructor called, Employee deleted')
```

```
In [6]: 1 obj=Employee()
```

Employee Class created

```
In [11]: 1 del obj
```

Destructor called, Employee deleted

```
In [2]: 1 def gen_fun():
2         yield 10
3         yield 20
4         yield 30
5         for i in gen_fun():
6             print(i)
```

10
20
30

```
In [3]: 1 class Disp:
2         var=10
3         def display(self):
4             print('In Class Method....')
5
6 obj=Disp()
7 print(obj.var)
8 obj.display()
```

```
10
In Class Method....
```

```
In [3]: 1 class Disp:
2         def __init__(self,val):
3             print('In Class Method....')
4             self.val=val
5             print('The value is:',self.val)
6
7 obj=Disp(10)
```

```
In Class Method....
The value is: 10
```

```
In [7]: 1 class Disp:
2         def __init__(self,var1,var2):
3             self.var1=var1
4             self.__var2=var2
5
6         def display(self):
7             print("From Class method, var1=",self.var1)
8             print("From Class method, var2=",self.__var2)
9
10 obj=Disp(10,20)
11 obj.display()
12 print('From Main Module, var1=',obj.var1)
13 print('From Main Module, var2=',obj.__var2)
```

```
From Class method, var1= 10
From Class method, var2= 20
From Main Module, var1= 10
```

```
-----
AttributeError                                Traceback (most recent call last)
Input In [7], in <cell line: 13>()
    11 obj.display()
    12 print('From Main Module, var1=',obj.var1)
--> 13 print('From Main Module, var2=',obj.__var2)

AttributeError: 'Disp' object has no attribute '__var2'
```

```
In [4]: 1 class Students:
2         def __init__(self,name):
3             self.name=name
4             self.marks=[]
5         def entermarks(self):
6             for i in range(3):
7                 m=int(input('Enter marks of %s in Subject %d:'%(self.name,i+1)))
8                 self.marks.append(m)
9         def display(self):
10            print(self.name,'got',self.marks)
```

```
In [5]: 1 s1=Students('Parth')
2        s1.entermarks()
```

Enter marks of Parth in Subject 1:23
Enter marks of Parth in Subject 2:24
Enter marks of Parth in Subject 3:20

```
In [1]: 1 s2=Students('Aakash')
2        s2.entermarks()
```

```
-----
NameError                                Traceback (most recent call last)
Input In [1], in <cell line: 1>()
----> 1 s2=Students('Aakash')
      2 s2.entermarks()

NameError: name 'Students' is not defined
```

```
In [6]: 1 s1.display()
2        s2.display()
```

Parth got [23, 24, 20]

```
-----
NameError                                Traceback (most recent call last)
Input In [6], in <cell line: 2>()
      1 s1.display()
----> 2 s2.display()

NameError: name 's2' is not defined
```

```
In [7]: 1 class String:
2         def __init__(self):
3             self.vowels=0
4             self.consonants=0
5             self.spaces=0
6             self.uppercase=0
7             self.lowercase=0
8             self.string=input("Enter a string:")
9         def count_uppercase(self):
10            for letter in self.string:
11                if letter.isupper():
12                    self.uppercase+=1
13        def count_lowercase(self):
14            for letter in self.string:
15                if letter.islower():
16                    self.lowercase+=1
17        def count_vowels(self):
18            for letter in self.string:
19                if letter in 'aeiouAEIOU':
20                    self.vowels+=1
21        def count_spaces(self):
22            for letter in self.string:
23                if letter.isspace():
24                    self.spaces+=1
25        def count_consonants(self):
26            for letter in self.string:
27                if (letter.isspace()) or (letter.isdigit()):
28                    pass
29                if (letter not in 'aeiouAEIOU'):
30                    self.consonants+=1
31        def compute_stats(self):
32            self.count_uppercase()
33            self.count_lowercase()
34            self.count_vowels()
35            self.count_consonants()
36            self.count_spaces()
37        def show_stats(self):
38            print('Vowels:%d'%self.vowels)
39            print('Consonants:%d'%self.consonants)
40            print('Spaces:%d'%self.spaces)
41            print('Uppercase:%d'%self.uppercase)
42            print('Lowercase:%d'%self.lowercase)
```

```
In [8]: 1 s=String()
2         s.compute_stats()
3         s.show_stats()
```

```
Enter a string:Hello This is a test STRing
Vowels:7
Consonants:20
Spaces:5
Uppercase:5
Lowercase:17
```


In [5]:

```
1 class Atm:
2     # constructor
3     def __init__(self):
4         #1print(id(self))
5         self.pin=''
6         self.balance=0
7         self.menu()
8     def menu(self):
9         user_input=input("""
10        Hi how can I help you?
11        1. Press 1 to create pin
12        2. Press 2 to change pin
13        3. Press 3 to check balance
14        4. Press 4 to withdraw
15        5. Anything else to exit
16        """)
17        if user_input=='1':
18            self.create_pin()
19        elif user_input=='2':
20            self.change_pin()
21        elif user_input == '3':
22            self.check_balance()
23        elif user_input=='4':
24            self.withdraw()
25        else:
26            exit()
27    def create_pin(self):
28        user_pin=input('enter your pin')
29        self.pin=user_pin
30        user_balance=int(input('enter balance'))
31        self.balance=user_balance
32        print('pin created successfully')
33        self.menu()
34    def change_pin(self):
35        old_pin=input('enter old pin')
36        if old_pin ==self.pin:
37            # Let him change the pin
38            new_pin=input('enter new pin')
39            self.pin=new_pin
40            print('pin change successful')
41            self.menu()
42        else:
43            print('enter correct pin')
44            self.menu()
45    def check_balance(self):
46        user_pin=input('enter your pin')
47        if user_pin==self.pin:
48            print('your balance is ',self.balance)
49        else:
50            print('enter correct pin')
51            self.menu()
52    def withdraw(self):
53        user_pin=input('enter the pin')
54        if user_pin==self.pin:
55            # allow to withdraw
56            amount=int(input('enter the amount'))
57            if amount<=self.balance:
58                self.balance=self.balance-amount
59                print('withdrawl successful.balance is',self.balance)
60            else:
61                print('increase your balance')
62        else:
63            print('enter correct pin')
```


In [6]:

```
1 sbi=Atm()
```


Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
1
enter your pin1234
enter balance5000
pin created successfully

Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
2
enter old pin1234
enter new pin0411
pin change successful

Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
3
enter your pin1234
enter correct pin

Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
3
enter your pin0411
your balance is 5000

Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
4
enter the pin0411
enter the amount6000
increase your balance

Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
4
enter the pin0411
enter the amount2500
withdrawl successful.balance is 2500

Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
s

```
In [1]: 1 class Roman:
2         def __init__(self,s):
3             self.s=s
4         def roman_to_int(self):
5             rom_val={'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000}
6             int_val=0
7             for i in range(len(self.s)):
8                 if i>0 and rom_val[self.s[i]]>rom_val[self.s[i-1]]:
9                     int_val+=rom_val[self.s[i]]-2*rom_val[self.s[i-1]]
10                else:
11                    int_val+=rom_val[self.s[i]]
12            print(int_val)
```

```
In [16]: 1 obj=Roman('MMXXIII')
```

```
In [17]: 1 obj.roman_to_int()
```

2023

```
In [1]: 1 rom_val={'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000}
2         int_val=0
3         s='IV'
4         for i in range(len(s)):
5             print(rom_val[s[i-1]])
```

5
1

In [20]:

```
1 class Atm:
2     # constructor
3     def __init__(self):
4         #1print(id(self))
5         self.pin=''
6         self.balance=0
7         self.menu()
8     def menu(self):
9         user_input=input("""
10        Hi how can I help you?
11        1. Press 1 to create pin
12        2. Press 2 to change pin
13        3. Press 3 to check balance
14        4. Press 4 to withdraw
15        5. Press 5 to deposit
16        6. Anything else to exit
17        """)
18        if user_input=='1':
19            self.create_pin()
20        elif user_input=='2':
21            self.change_pin()
22        elif user_input == '3':
23            self.check_balance()
24        elif user_input=='4':
25            self.withdraw()
26        elif user_input=='5':
27            self.deposit()
28        else:
29            exit()
30    def create_pin(self):
31        user_pin=input('enter your pin')
32        self.pin=user_pin
33        user_balance=int(input('enter balance'))
34        self.balance=user_balance
35        print('pin created successfully')
36        self.menu()
37    def change_pin(self):
38        old_pin=input('enter old pin')
39        if old_pin ==self.pin:
40            # let him change the pin
41            new_pin=input('enter new pin')
42            self.pin=new_pin
43            print('pin change successful')
44            self.menu()
45        else:
46            print('enter correct pin')
47            self.menu()
48    def check_balance(self):
49        user_pin=input('enter your pin')
50        if user_pin==self.pin:
51            print('your balance is ',self.balance)
52        else:
53            print('enter correct pin')
54            self.menu()
55    def withdraw(self):
56        user_pin=input('enter the pin')
57        if user_pin==self.pin:
58            # allow to withdraw
59            amount=int(input('enter the amount'))
60            if amount<=self.balance:
61                self.balance=self.balance-amount
62                print('withdrawl successful.balance is',self.balance)
63            else:
64                print('increase your balance')
65        else:
```

```

66         print('enter correct pin')
67     self.menu()
68     def deposit(self):
69         user_pin=input('Enter pin:')
70         if user_pin==self.pin:
71             dep=int(input('Enter amount to deposit:'))
72             self.balance+=dep
73         else:
74             print('enter correct pin')
75     self.menu()

```

In [21]: 1 sbi=Atm()

```

Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Press 5 to deposit
6. Anything else to exit
1
enter your pin0411
enter balance2500
pin created successfully

```

```

Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Press 5 to deposit
6. Anything else to exit
5
Enter pin:0411
Enter amount to deposit:500

```

```

Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Press 5 to deposit
6. Anything else to exit
3
enter your pin0411
your balance is 3000

```

```

Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Press 5 to deposit
6. Anything else to exit
8

```

In []: 1 num=int(input('Enter numerator'))

In [1]: 1 #Zero Division Error

```
In [1]: 1 num=int(input('Enter numerator: '))
2 deno=int(input('Enter denominator: '))
3 try:
4     quo=num/deno
5     print('QUOTIENT: ',quo)
6 except ZeroDivisionError:
7     print('Denominator cannot be zero')
8 except Exception as e:
9     print(e)
```

Enter numerator: 25
Enter denominator: 0
Denominator cannot be zero

```
In [6]: 1 num=int(input('Enter numerator: '))
2 deno=int(input('Enter denominator: '))
3 quo=num/deno
4 print('QUOTIENT: ',quo)
```

Enter numerator: 10
Enter denominator: 0

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Input In [6], in <cell line: 3>()
      1 num=int(input('Enter numerator: '))
      2 deno=int(input('Enter denominator: '))
----> 3 quo=num/deno
      4 print('QUOTIENT: ',quo)

ZeroDivisionError: division by zero
```

```
In [4]: 1 #Multiple except blocks:
2 try:
3     num=int(input('Enter the number: '))
4     print(num**2)
5 except (KeyboardInterrupt):
6     print('Keyboard interruption occurred')
7 except (ValueError):
8     print('Please Check the Value')
```

Keyboard interruption occurred

```
In [1]: 1 try:
2     num=int(input('Enter the number: '))
3     print(num**2)
4 except (KeyboardInterrupt):
5     print('Error occurred')
```

Error occurred

1. ArithmeticError
2. AttributeError
3. Exception
4. EOFError
5. ImportError
6. IndentationError
7. IndexError
8. KeyError
9. NameError

- 10. SyntaxError
- 11. TypeError
- 12. ValueError
- 13. ZeroDivisionError

In [10]: 1 *# try except else*

```
In [12]: 1 try:
2     num=int(input('Enter value for denominator:'))
3     div=10/num
4     print(div)
5 except ZeroDivisionError:
6     print('Cannot be divisible by zero')
7 else:
8     print('Successful..')
```

Enter value for denominator:2
5.0
Successful..

In [13]: 1 *#raise an exception*

```
In [14]: 1 try:
2     num=10
3     print(num)
4     raise ValueError
5 except:
6     print('Exception occurred...')
```

10
Exception occurred...

```
In [1]: 1 try:
2     raise NameError
3 except:
4     print('Re-raising Error')
5     raise
```

Re-raising Error

```
-----
NameError                                Traceback (most recent call last)
Input In [1], in <cell line: 1>()
      1 try:
----> 2     raise NameError
      3 except:
      4     print('Re-raising Error')

NameError:
```

```
In [18]: 1 def divide():
2         try:
3             num=int(input('Enter value for denominator:'))
4             div=10/num
5             print(div)
6         except ZeroDivisionError:
7             print('Cannot be divisible by zero')
8         else:
9             print('Successful..')
10 divide()
```

Enter value for denominator:0
Cannot be divisible by zero

```
In [8]: 1 try:
2         num=int(input('Enter value for denominator:'))
3         div=10/num
4         print(div)
5     except Exception as e:
6         print(e)
```

Enter value for denominator:0
division by zero

```
In [10]: 1 3+3.0
```

Out[10]: 6.0

```
In [11]: 1 i=1
2         i.append(2)
```

```
-----
AttributeError                                Traceback (most recent call last)
Input In [11], in <cell line: 2>()
      1 i=1
----> 2 i.append(2)

AttributeError: 'int' object has no attribute 'append'
```

```
In [12]: 1 int('a')
```

```
-----
ValueError                                    Traceback (most recent call last)
Input In [12], in <cell line: 1>()
----> 1 int('a')

ValueError: invalid literal for int() with base 10: 'a'
```

```
In [13]: 1 try:
2         i=['a','b']
3         t=i[1]
4         print('1')
5     except Exception:
6         print("2")
7     print('3')
```

1
3


```
In [3]: 1 def a():
2         try:
3             f(x,4)
4         finally:
5             print('after f')
6         print('1')
7     a()
```

after f

```
-----
NameError                                Traceback (most recent call last)
Input In [3], in <cell line: 7>()
      5         print('after f')
      6         print('1')
----> 7     a()

Input In [3], in a()
      1 def a():
      2     try:
----> 3         f(x,4)
      4     finally:
      5         print('after f')

NameError: name 'f' is not defined
```

```
In [4]: 1 def display(n):
2         while True:
3             try:
4                 n+=1
5                 if n==21:
6                     raise StopIteration
7             except StopIteration:
8                 print('stop')
9                 break
10            else:
11                print(n,end=" ")
12
13     display(1)
```

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 stop

In [1]:

```
1 class Pizza:
2     def __init__(self, size, toppings, cheese):
3         self.size = size
4         self.toppings = toppings
5         self.cheese = cheese
6     def price(self):
7         self.cost = 0
8         if self.size == 'small':
9             self.cost += 50
10        elif self.size == 'medium':
11            self.cost += 100
12        else:
13            self.cost += 200
14        topping_prices_20 = ['corn', 'tomato', 'onion', 'capsicum']
15        topping_prices_50 = ['mushroom', 'olives', 'broccoli']
16        for topping in self.toppings:
17            if topping in topping_prices_20:
18                self.cost += 20
19            else: self.cost += 50
20        #for cheese
21        self.cost += 50 * len(self.cheese)
22        return self.cost
23
24 class Order:
25     def __init__(self, name, customerid):
26         self.name = name
27         self.customerid = customerid
28
29     def order(self, n):
30         self.pizzas = []
31         for i in range(n):
32             toppings = []
33             cheese = []
34             print('Customize Pizza',i+1)
35             size = input('Select size: ')
36             t = int(input('How many toppings: '))
37             for i in range(t):
38                 toppings.append(input('Enter toppings: '))
39             t = int(input('How many cheese: '))
40             for i in range(t):
41                 cheese.append(input('Enter cheese: '))
42             self.pizzas.append(Pizza(size, toppings, cheese))
43
44     def bill(self):
45         self.total = 0
46         count = 1
47         for p in self.pizzas:
48             print('Pizza', count)
49             print("Size:",p.size)
50             print("Toppings:",p.toppings)
51             print("Cheese:",p.cheese)
52             self.total += p.price()
53             count += 1
54         print('Total bill amount:', self.total)
55
56 number=int(input("How many pizzas you want to order: "))
57 order1=Order('Parth',1)
58 order1.order(number)
59 order1.bill()
```

How many pizzas you want to order: 1
Customize Pizza 1
Select size: medium
How many toppings: 2
Enter toppings: tomato
Enter toppings: brocolli
How many cheese: 1
Enter cheese: mozzarella
Pizza 1
Size: medium
Toppings: ['tomato', 'brocolli']
Cheese: ['mozzarella']
Total bill amount: 220

In [2]:

```
1 class Point:
2     def __init__(self, initX, initY):
3         self.x = initX
4         self.y = initY
5     def getX(self):
6         return self.x
7     def getY(self):
8         return self.y
9     def distanceFromOrigin(self):
10        return ((self.x ** 2) + (self.y ** 2)) ** 0.5
11    def __str__(self):
12        return "x = {}, y = {}".format(self.x, self.y)
13 p = Point(7,6)
14 print(p)
```

x = 7, y = 6

In [3]:

```
1 class User:
2     def __init__(self,id,name):
3         self.id=id
4         self.name=name
5     def display_user(self):
6         print('ID',self.id,'Name:',self.name)
7
8 class Item:
9     def __init__(self,id,name,price,sold,available):
10        self.id=id
11        self.name=name
12        self.price=price
13        self.sold=sold
14        self.available=available
15
16 class Cart:
17     def __init__(self,user):
18         self.user=user
19         self.cart_items=[]
20     def insert_items(self,item,quantity):
21         for i in range(quantity):
22             if item.available==0:
23                 print('Out of stock')
24                 break
25             self.cart_items.append(item)
26             item.sold+=1
27             item.available-=1
28     def display_cart(self):
29         print('This Cart belongs to',self.user.name,'with ID',self.user.id)
30         self.total=0
31         for i in self.cart_items:
32             print('Item',i.name)
33             self.total+=i.price
34         print('Total price = ',self.total)
35
36 #Creating a user
37 user1=User(1,'Parth')
38 user1.display_user()
39 #Creating items
40 apple=Item(1,'apple',100,0,10)
41 greencoconut=Item(2,'green coconut',150,0,1)
42 milk=Item(3,'milk',24,0,100)
43 #Creating two carts for user1
44 cart1=Cart(user1)
45 cart2=Cart(user1)
46 #Adding items to cart1
47 cart1.insert_items(apple,2)
48 cart1.insert_items(milk,3)
49 #Adding items to cart2
50 cart2.insert_items(greencoconut,3)
51 cart2.insert_items(milk,20)
52 #Displaying details of cart1 and cart2
53 cart1.display_cart()
54 cart2.display_cart()
```

[illegible]

In [4]:

```
1 # This is simplest Student data management program in python
2
3 # Create class "Student"
4 class Student:
5
6     # Constructor
7     def __init__(self, name, rollno, m1, m2):
8         self.name = name
9         self.rollno = rollno
10        self.m1 = m1
11        self.m2 = m2
12
13    # Function to create and append new student
14    def accept(self, Name, Rollno, marks1, marks2):
15
16    # use ' int(input()) ' method to take input from user
17        ob = Student(Name, Rollno, marks1, marks2)
18        ls.append(ob)
19
20    # Function to display student details
21    def display(self, ob):
22        print("Name : ", ob.name)
23        print("RollNo : ", ob.rollno)
24        print("Marks1 : ", ob.m1)
25        print("Marks2 : ", ob.m2)
26        print("\n")
27
28    # Search Function
29    def search(self, rn):
30        for i in range(ls.__len__()):
31            if(ls[i].rollno == rn):
32                return i
33
34    # Delete Function
35    def delete(self, rn):
36        i = obj.search(rn)
37        del ls[i]
38
39    # Update Function
40    def update(self, rn, No):
41        i = obj.search(rn)
42        roll = No
43        ls[i].rollno = roll
44
45
46    # Create a list to add Students
47    ls = []
48    # an object of Student class
49    obj = Student('', 0, 0, 0)
50
51    print("\nOperations used, ")
52    print("\n1.Accept Student details\n2.Display Student Details\n3.Search Details of
53
54    # ch = int(input("Enter choice:"))
55    # if(ch == 1):
56    obj.accept("A", 1, 100, 100)
57    obj.accept("B", 2, 90, 90)
58    obj.accept("C", 3, 80, 80)
59
60    # elif(ch == 2):
61    print("\n")
62    print("\nList of Students\n")
63    for i in range(ls.__len__()):
64        obj.display(ls[i])
65
```

```
66 # elif(ch == 3):
67 print("\n Student Found, ")
68 s = obj.search(2)
69 obj.display(ls[s])
70
71 # elif(ch == 4):
72 obj.delete(2)
73 print(ls.__len__())
74 print("List after deletion")
75 for i in range(ls.__len__()):
76     obj.display(ls[i])
77
78 # elif(ch == 5):
79 obj.update(3, 2)
80 print(ls.__len__())
81 print("List after updation")
82 for i in range(ls.__len__()):
83     obj.display(ls[i])
84
85 # else:
86 print("Thank You !")
87
```


Operations used,

- 1.Accept Student details
- 2.Display Student Details
- 3.Search Details of a Student
- 4.Delete Details of Student
- 5.Update Student Details
- 6.Exit

List of Students

Name : A
RollNo : 1
Marks1 : 100
Marks2 : 100

Name : B
RollNo : 2
Marks1 : 90
Marks2 : 90

Name : C
RollNo : 3
Marks1 : 80
Marks2 : 80

Student Found,

Name : B
RollNo : 2
Marks1 : 90
Marks2 : 90

2

List after deletion

Name : A
RollNo : 1
Marks1 : 100
Marks2 : 100

Name : C
RollNo : 3
Marks1 : 80
Marks2 : 80

2

List after updation

Name : A
RollNo : 1
Marks1 : 100
Marks2 : 100

Name : C
RollNo : 2
Marks1 : 80

Marks2 : 80

Thank You !

In [7]:

```
1 class Student:
2     def __init__(self, name, roll_no, enrollment_no, branch, email, contact_no):
3         self.name = name
4         self.roll_no = roll_no
5         self.enrollment_no = enrollment_no
6         self.branch = branch
7         self.email = email
8         self.contact_no = contact_no
9         self.registered_events = []
10
11     def display_details(self):
12         print(f"\nStudent Details:")
13         print(f"Name: {self.name}")
14         print(f"Roll No: {self.roll_no}")
15         print(f"Enrollment No: {self.enrollment_no}")
16         print(f"Branch: {self.branch}")
17         print(f>Email: {self.email}")
18         print(f>Contact No: {self.contact_no}")
19
20     def display_registered_events(self):
21         if self.registered_events:
22             print(f"\nRegistered Events for {self.name} ({self.enrollment_no}):")
23             for event in self.registered_events:
24                 print(f"- {event}")
25         else:
26             print(f"\n{self.name} ({self.enrollment_no}) has not registered for
27
28
29 class EventSelection:
30     def __init__(self, student):
31         self.student = student
32         self.group_events = ["Cricket", "Football", "Basketball", "Volleyball"]
33         self.individual_events = ["Sprint Race", "Table Tennis"]
34         self.max_group_events = 2
35
36     def display_available_events(self):
37         print("\nAvailable Events:")
38         print("Group Events:")
39         for event in self.group_events:
40             print(f"- {event}")
41         print("\nIndividual Events:")
42         for event in self.individual_events:
43             print(f"- {event}")
44
45     def select_event(self, event_name):
46         if event_name in self.group_events:
47             if len(self.student.registered_events) < self.max_group_events:
48                 self.student.registered_events.append(event_name)
49                 print(f"{event_name} selected successfully!")
50             else:
51                 print(f"Error: Maximum limit of {self.max_group_events} group ev
52         elif event_name in self.individual_events:
53             self.student.registered_events.append(event_name)
54             print(f"{event_name} selected successfully!")
55         else:
56             print(f"Error: {event_name} is not a valid event.")
57
58
59 # Interactive menu
60 def display_menu():
61     print("\nMenu:")
62     print("1. Register a new student")
63     print("2. Register for events")
64     print("3. Search by Enrollment No")
65     print("4. Exit")
```

```

66
67
68 def register_new_student():
69     name = input("Enter student name: ")
70     roll_no = input("Enter roll number: ")
71     enrollment_no = input("Enter enrollment number: ")
72     branch = input("Enter branch: ")
73     email = input("Enter email: ")
74     contact_no = input("Enter contact number: ")
75
76     student = Student(name, roll_no, enrollment_no, branch, email, contact_no)
77     student.display_details()
78     return student
79
80
81 def register_for_events(students):
82     enrollment_no = input("Enter enrollment number to register for events: ")
83     student_found = False
84
85     for student in students:
86         if student.enrollment_no == enrollment_no:
87             student_found = True
88             event_selection = EventSelection(student)
89             event_selection.display_available_events()
90
91             while True:
92                 event_name = input("\nEnter an event to select (or 'done' to fin
93                 if event_name.lower() == 'done':
94                     break
95                 event_selection.select_event(event_name)
96
97             student.display_registered_events()
98             break
99
100     if not student_found:
101         print("Student not found.")
102
103
104 def search_by_enrollment(students):
105     enrollment_no = input("Enter enrollment number to search: ")
106     student_found = False
107
108     for student in students:
109         if student.enrollment_no == enrollment_no:
110             student_found = True
111             student.display_details()
112             student.display_registered_events()
113             break
114
115     if not student_found:
116         print("Student not found.")
117
118
119 # Main program loop
120 students = []
121
122 while True:
123     display_menu()
124     choice = input("Enter your choice (1-4): ")
125
126     if choice == '1':
127         student = register_new_student()
128         students.append(student)
129     elif choice == '2':
130         register_for_events(students)
131     elif choice == '3':

```

```
132         search_by_enrollment(students)
133     elif choice == '4':
134         print("Exiting the program.")
135         break
136     else:
137         print("Invalid choice. Please enter a number between 1 and 4.")
138
```

Menu:

```
1. Register a new student
2. Register for events
3. Search by Enrollment No
4. Exit
Enter your choice (1-4): 1
Enter student name: Parth
Enter roll number: 1
Enter enrollment number: 123
Enter branch: CSE
Enter email: parth@ljku.edu.in
Enter contact number: 1234567890
```

Student Details:

```
Name: Parth
Roll No: 1
Enrollment No: 123
Branch: CSE
Email: parth@ljku.edu.in
Contact No: 1234567890
```

Menu:

```
1. Register a new student
2. Register for events
3. Search by Enrollment No
4. Exit
Enter your choice (1-4): 4
Exiting the program.
```